Architecture design patterns that support security

When you design workload architectures, you should use industry patterns that address common challenges. Patterns can help you make intentional tradeoffs within workloads and optimize for your desired outcome. They can also help mitigate risks that originate from specific problems, which can affect reliability, performance, cost, and operations. Those risks might be indicative of lack of security assurances, if left unattended can pose significant risks to the business. These patterns are backed by real-world experience, are designed for cloud scale and operating models, and are inherently vendor agnostic. Using well-known patterns as a way to standardize your workload design is a component of operational excellence.

Many design patterns directly support one or more architecture pillars. Design patterns that support the Security pillar prioritize concepts like segmentation and isolation, strong authorization, uniform application security, and modern protocols.

The following table summarizes Architecture design patterns that support the goals of security.

Expand table

Pattern	Summary
Ambassador	Encapsulates and manages network communications by offloading cross-cutting tasks that are related to network communication. The resulting helper services initiate communication on behalf of the client. This mediation point provides an opportunity to augment security on network communications.
Backends for Frontends	Individualizes the service layer of a workload by creating separate services that are exclusive to a specific frontend interface. Because of this separation, the security and authorization in the service layer that support one client can be tailored to the functionality provided by that client, potentially reducing the surface area of an API and lateral movement among different backends that might expose different capabilities.
Bulkhead	Introduces intentional and complete segmentation between components to isolate the blast radius of malfunctions. You can also use this strategy to contain security incidents to the compromised bulkhead.
Claim Check	Separates data from the messaging flow, providing a way to separately retrieve the data related to a message. This pattern supports keeping sensitive data out of message bodies, instead keeping it managed in a secured data store. This configuration enables you to establish stricter authorization to support access to the sensitive data from services that are expected to use the data, but remove visibility from ancillary services like queue monitoring solutions.

Pattern	Summary
Federated Identity	Delegates trust to an identity provider that's external to the workload for managing users and providing authentication for your application. By externalizing user management and authentication, you can get evolved capabilities for identity-based threat detection and prevention without needing to implement these capabilities in your workload. External identity providers use modern interoperable authentication protocols.
Gatekeeper	Offloads request processing that's specifically for security and access control enforcement before and after forwarding the request to a backend node. Adding a gateway into the request flow enables you to centralize security functionality like web application firewalls, DDoS protection, bot detection, request manipulation, authentication initiation, and authorization checks.
Gateway Aggregation	Simplifies client interactions with your workload by aggregating calls to multiple backend services in a single request. This topology often reduces the number of touch points a client has with a system, which reduces the public surface area and authentication points. The aggregated backends can stay fully network-isolated from clients.
Gateway Offloading	Offloads request processing to a gateway device before and after forwarding the request to a backend node. Adding a gateway into the request flow enables you to centralize security functionality like web application firewalls and TLS connections with clients. Any offloaded functionality that's platform-provided already offers enhanced security.
Publisher/Subscriber	Decouples components in an architecture by replacing direct client-to-service communication with communication via an intermediate message broker or event bus. This replacement introduces an important security segmentation boundary that enables queue subscribers to be network-isolated from the publisher.
Quarantine	Ensures external assets meet a team-agreed quality level before being authorized to consume them in the workload. This check serves as a first security validation of external artifacts. The validation on an artifact is conducted in a segmented environment before it's used within the secure development lifecycle (SDL).
Sidecar	Extends the functionality of an application by encapsulating nonprimary or cross-cutting tasks in a companion process that exists alongside the main application. By encapsulating these tasks and deploying them out-of-process, you can reduce the surface area of sensitive processes to only the code that's needed to accomplish the task. You can also use sidecars to add cross-cutting security controls to an application component that's not natively designed with that functionality.
Throttling	Imposes limits on the rate or throughput of incoming requests to a resource or component. You can design the limits to help prevent resource exhaustion that could result from automated abuse of the system.
Valet Key	Grants security-restricted access to a resource without using an intermediary resource to proxy the access. This pattern enables a client to directly access a

Pattern	Summary
	resource without needing long-lasting or standing credentials. All access requests start with an auditable transaction. The granted access is then limited in both scope and duration. This pattern also makes it easier to revoke the granted access.

Next steps

Review the Architecture design patterns that support the other Azure Well-Architected Framework pillars:

- Architecture design patterns that support reliability
- Architecture design patterns that support cost optimization
- Architecture design patterns that support operational excellence
- Architecture design patterns that support performance efficiency

---

Last updated on 08/29/2025